

WEB4 PHP – Game of Thrones Quotes

SS 2026

Aufwand in h: 10

Nathalie Sonnleitner, Tim Peko

s2420458029

Inhaltsverzeichnis

1. Einleitung	2
2. Projektmitglieder	2
3. Anforderungsübersicht	2
3.1. Rollenmodell	2
4. Datenmodell	3
4.1. ER-Diagramm	3
4.2. Tabellen	3
4.2.1. users	3
4.2.2. quotes	3
4.2.3. comments	4
4.3. Testdaten	4
5. Architektur	5
5.1. MVC-Schichten	5
5.2. Verzeichnisstruktur	5
5.3. Request-Flow (Beispiel: Kommentar erstellen)	6
6. Sicherheitskonzept	6
7. Testfälle	6
7.1. Übersicht	6
7.2. Detailergebnisse	6
8. Installation	7
8.1. Voraussetzungen	7
8.2. Schritte	7
8.3. Entwicklung mit mise	8

1. Einleitung

Diese Dokumentation beschreibt die Webanwendung **GoT Quotes** – eine PHP-Anwendung im MVC-Pattern, die berühmte Game-of-Thrones-Zitate präsentiert und es eingeloggten Nutzern ermöglicht, Kommentare zu verfassen, zu bearbeiten und zu löschen. Administratoren verwalten Benutzer und Zitate.

Start-URL (XAMPP): `http://localhost/public/` – Document Root muss auf das Verzeichnis `public/` zeigen.

Start-URL (Entwicklung mit mise): Apache auf `public/` mit MySQL Port `33060` (siehe `mise.toml`).

2. Projektmitglieder

Name	Matrikelnummer
Nathalie Sonnleitner	—
Tim Peko	s2420458029

3. Anforderungsübersicht

Die Anwendung erfüllt die Pflichtanforderungen der Projektangabe:

- MVC-Architektur mit PHP 8.x und PDO
- Rollenbasierte Interaktion (Gast, User, Admin)
- Registrierung und Login (Session-basiert, kein Auto-Login nach Registrierung)
- Vollständiges CRUD für Kommentare (Haupt-Ressource) und Zitate (Admin)
- Benutzerverwaltung für Admins (Löschen, Rolle ändern)
- Serverseitige Validierung, SQL-Injection- und XSS-Schutz
- Valides HTML5, CSS-Layout, keine JS-Validierung

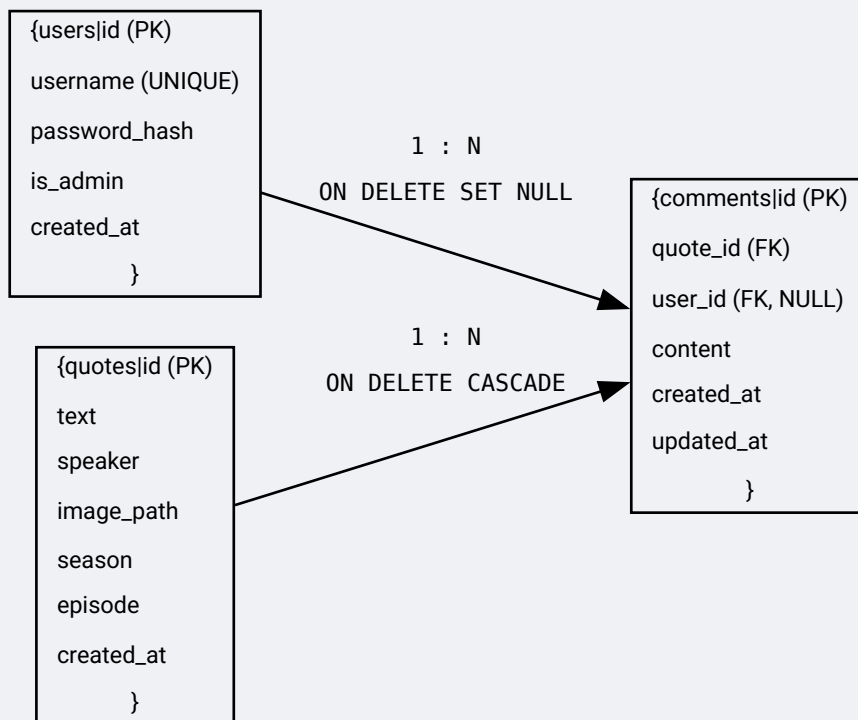
3.1. Rollenmodell

Aktion	Gast	User	Admin
Zitate & Kommentare lesen	✓	✓	✓
Kommentar schreiben / eigenes bearbeiten & löschen	—	✓	✓
Fremden Kommentar löschen	—	—	✓
Fremden Kommentar bearbeiten	—	—	X
Zitate verwalten (CRUD)	—	—	✓
Benutzerverwaltung	—	—	✓

Bei gelöschten Benutzern bleiben Kommentare erhalten; `user_id` wird auf `NULL` gesetzt und in der UI als graues `<deleted>` angezeigt.

4. Datenmodell

4.1. ER-Diagramm



4.2. Tabellen

4.2.1. users

Spalte	Typ	Constraints
id	INT UNSIGNED	PK, AUTO_INCREMENT
username	VARCHAR(50)	NOT NULL, UNIQUE
password_hash	VARCHAR(255)	NOT NULL
is_admin	TINYINT(1)	NOT NULL, DEFAULT 0
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

4.2.2. quotes

Spalte	Typ	Constraints
id	INT UNSIGNED	PK, AUTO_INCREMENT
text	TEXT	NOT NULL
speaker	VARCHAR(100)	NOT NULL
image_path	VARCHAR(255)	NULL – optional, derzeit ungenutzt
season / episode	TINYINT UNSIGNED	NULL – optional
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

4.2.3. comments

Spalte	Typ	Constraints
id	INT UNSIGNED	PK, AUTO_INCREMENT
quote_id	INT UNSIGNED	FK → quotes.id, ON DELETE CASCADE
user_id	INT UNSIGNED	FK → users.id, NULL, ON DELETE SET NULL
content	TEXT	NOT NULL
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP
updated_at	DATETIME	NULL ON UPDATE

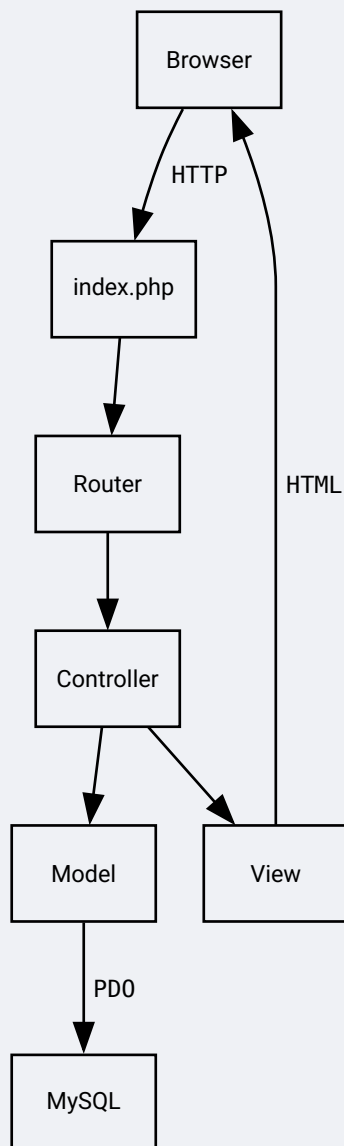
4.3. Testdaten

SQL-Dump: `doc/team_4.sql`

- Admin: `admin` / `admin`
- Testuser: `tyrion_fan`, `arya_fan` – Passwort: `password123`
- 12 Zitate, 15 Kommentare (ohne Bilder)

5. Architektur

5.1. MVC-Schichten



5.2. Verzeichnisstruktur

1	public/	
2	— index.php	# Front Controller
3	— .htaccess	# URL Rewriting
4	— assets/css/app.css	
5	— src/	
6	— bootstrap.php	
7	— config.php	
8	— Router.php	
9	— Controller/	# Auth, Quote, Comment, Admin/*
10	— Model/	# User, Quote, Comment
11	— Service/	# AuthService, ValidationService

5.3. Request-Flow (Beispiel: Kommentar erstellen)

1. `POST /quotes/{id}/comments` → Router
2. `CommentController::store` — CSRF prüfen, Login erzwingen
3. `ValidationService::commentContent` — serverseitige Validierung
4. `Comment::create` — Prepared Statement mit Session- `user_id`
5. Redirect zur Zitat-Detailseite mit Flash-Message

6. Sicherheitskonzept

Bedrohung	Maßnahme
SQL Injection	PDO Prepared Statements für alle Queries mit User-Input
XSS	<code>htmlspecialchars()</code> bei jeder Ausgabe usergenerierter Inhalte
Session Fixation	<code>session_regenerate_id()</code> nach Login
CSRF	Token in allen POST-Formularen, Validierung in Controllern
IDOR	Berechtigungsprüfung: Kommentar-Bearbeitung nur für Autor
Passwörter	<code>password_hash()</code> / <code>password_verify()</code> , <code>PASSWORD_DEFAULT</code>

7. Testfälle

Alle Tests wurden manuell in Google Chrome unter XAMPP durchgeführt.

7.1. Übersicht

Testsuite	Tests	Fehler
Total	26	0
Authentifizierung	6	0
Zitate & Kommentare	9	0
Administration	8	0
Sicherheit	3	0

7.2. Detailergebnisse

Testname	Status	Zeit
Authentifizierung		
T-AUTH-01: Registrierung gültig → Erfolg, Redirect Login	✓ Erfolgreich	manuell
T-AUTH-02: Duplicate Username → Fehler	✓ Erfolgreich	manuell
T-AUTH-03: Passwort < 8 Zeichen → Fehler	✓ Erfolgreich	manuell
T-AUTH-04: Login korrekt → Session aktiv	✓ Erfolgreich	manuell

Testname	Status	Zeit
Authentifizierung		
T-AUTH-05: Login falsch → generische Fehlermeldung	✓ Erfolgreich	manuell
T-AUTH-06: Logout → Session beendet	✓ Erfolgreich	manuell
Zitate & Kommentare		
T-QUOTE-01: Gast sieht Liste und Detail	✓ Erfolgreich	manuell
T-CMT-01: Gast sieht Kommentare, kein Formular	✓ Erfolgreich	manuell
T-CMT-02: User erstellt Kommentar mit Username	✓ Erfolgreich	manuell
T-CMT-03: User bearbeitet eigenen Kommentar	✓ Erfolgreich	manuell
T-CMT-04: User löscht eigenen Kommentar	✓ Erfolgreich	manuell
T-CMT-05: Fremdes Edit → 403	✓ Erfolgreich	manuell
T-CMT-06: Fremdes Delete → 403	✓ Erfolgreich	manuell
T-CMT-07: Leerer Kommentar → Validierungsfehler	✓ Erfolgreich	manuell
T-CMT-08: XSS-Payload → escaped in Ausgabe	✓ Erfolgreich	manuell
Administration		
T-ADM-01: Admin löscht fremden Kommentar	✓ Erfolgreich	manuell
T-ADM-02: Admin kann fremden Kommentar nicht bearbeiten	✓ Erfolgreich	manuell
T-ADM-03: Admin togglet User-Rolle	✓ Erfolgreich	manuell
T-ADM-04: Admin löscht User → Kommentare zeigen <deleted>	✓ Erfolgreich	manuell
T-ADM-05: Nicht-Admin auf /admin → 403	✓ Erfolgreich	manuell
T-ADM-06: Letzter Admin nicht degradierbar	✓ Erfolgreich	manuell
T-ADM-07: Admin CRUD Zitate	✓ Erfolgreich	manuell
T-ADM-08: Admin kann eigene Rolle nicht entziehen	✓ Erfolgreich	manuell
Sicherheit		
T-SEC-01: SQL-Injection Login → kein Bypass	✓ Erfolgreich	manuell
T-SEC-02: Direktaufruf fremdes Edit → 403	✓ Erfolgreich	manuell
T-SEC-03: Ungültige Quote-ID → 404	✓ Erfolgreich	manuell

8. Installation

8.1. Voraussetzungen

- XAMPP (Apache + PHP 8.x + MySQL/MariaDB)
- Optional: mise für lokale Entwicklung

8.2. Schritte

1. ZIP entpacken
2. In phpMyAdmin `doc/team_4.sql` importieren
3. Apache DocumentRoot auf `public/` setzen
4. Anwendung unter `http://localhost/` aufrufen
5. Mit `admin` / `admin` einloggen

8.3. Entwicklung mit mise

```
1 mise run run:db          # MySQL starten (Port 33060)
2 mise run build:documentation
3 mise run package         # Abgabe-ZIP erzeugen
```

Datenbank-Credentials: siehe `mise.toml` – DB `team_4`, User `fh_webphp`.